

VMA Calculator — Technical Documentation

Version: 1.0 | **Date:** 2026-03-16 | **Maintainer:** Maryam Karim (maryamkarimbac@gmail.com, +224 702 49 12 30)

Table of Contents

- 1. [Project Overview](#)
- 2. [Architecture](#)
- 3. [VMA Calculation Logic](#)
- 4. [Authentication System](#)
- 5. [API Endpoints](#)
- 6. [Email Service](#)
- 7. [Database Schema](#)
- 8. [Key Frontend Components](#)
- 9. [Deployment Architecture](#)
- 10. [Deployment Configuration Details](#)
- 11. [Deployment Steps](#)
- 12. [Security Considerations](#)
- 13. [Troubleshooting](#)
- 14. [Developer Credits](#)

1. Project Overview

1.1 What the App Does

The VMA Calculator is a full-stack application that records Cooper (12-minute) and Demi-Cooper (6-minute) endurance tests and computes **Vitesse Maximale Aérobie (VMA)**. It supports:

- Real-time calculations for guests with transparent step-by-step breakdowns.
- Authenticated workflows where coaches or athletes can persist results, explore dashboards, export reports (CSV/PDF), and monitor individual progress.
- Automated welcome and password-reset emails (Mailjet) once the provider account is unblocked.

1.2 Guest vs Authenticated Flows

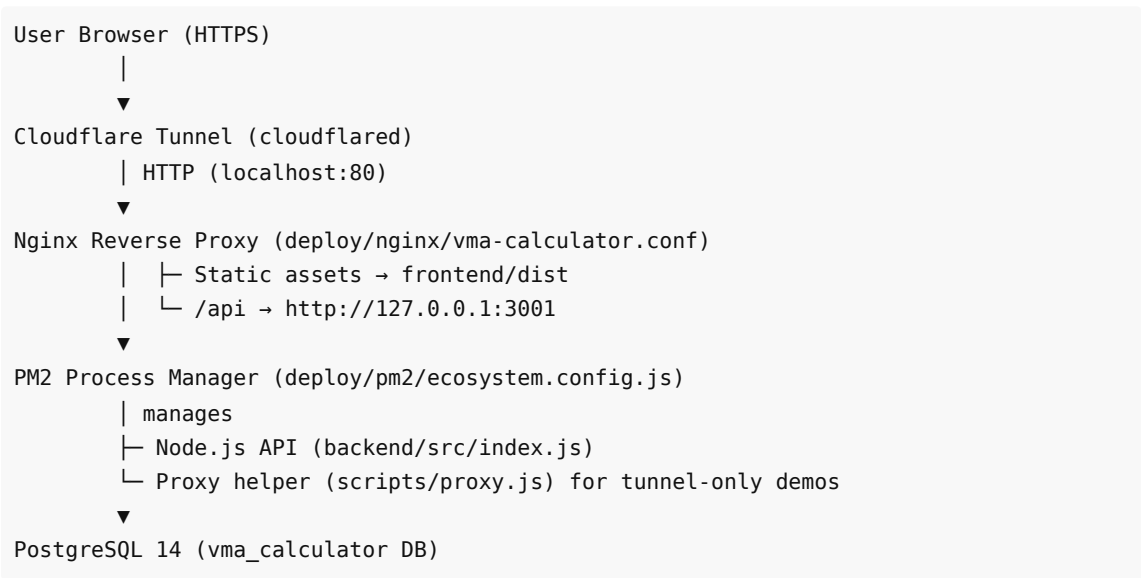
Flow	Guest	Authenticated User
Calculate VMA	✔ Via POST /api/tests without token (handled by optionalAuth)	✔ Same endpoint; result is persisted thanks to req.user.
Save Results	× CTA invites signup (TestForm.jsx).	✔ Records row in tests table with user_id.
View History	×	✔ /results, /results/:id, /dashboard, /profile.
Exports	×	✔ CSV (/api/tests/export/csv) + PDF via frontend utilities.
Password Reset	×	✔ ForgotPassword.jsx & ResetPassword.jsx use Mailjet templates.

1.3 Key Features (implemented)

- **Responsive SPA** built with React 18, Vite, Tailwind, lucide-react icons (`frontend/src`).
- **Rich test entry form** (`TestForm.jsx`) with live VMA preview, validation, and success states for guests/auth users.
- **Data management suite**: filterable results table (`Results.jsx`), detail view with PDF export (`TestDetail.jsx`), dashboards with Recharts visualizations (`Dashboard.jsx`), and profile summary widgets (`Profile.jsx`).
- **Secure API**: Express routes for auth, tests, statistics, and CSV export; `optionalAuth` allows seamless guest calculations while protecting persistence paths.
- **Email workflows**: `emailService.js` delivers welcome/reset emails through Mailjet or logs them locally when credentials are missing.
- **Deployment helpers**: `scripts/proxy.js` to serve SPA + proxy `/api` , and `scripts/start_tunnel.sh` to build/start backend/proxy/Cloudflare tunnel in one command.

2. Architecture

2.1 System Diagram



2.2 Frontend Structure (`frontend/`)

Path	Responsibility
<code>src/main.jsx</code>	Boots React with BrowserRouter, ThemeProvider, AuthProvider.
<code>src/App.jsx</code>	Defines route map (public vs <ProtectedRoute>).
<code>src/context/AuthContext.jsx</code>	Stores user session, exposes login, register, logout, auto-fetches <code>/api/auth/me</code> .
<code>src/context/ThemeContext.jsx</code>	Dark-mode toggle persisted in <code>localStorage</code> .
<code>src/pages/*</code>	Feature screens (TestSelection, TestForm, Results, TestDetail, Dashboard, Profile, Auth pages).

<code>src/components/*</code>	Shared UI such as Header, Footer, LevelBadge, Toast, ProtectedRoute, LoadingSpinner.
<code>src/services/api.js</code>	Fetch wrapper that injects JWT header, handles CSV, exposes typed methods per backend route.
<code>src/utls/vma.js</code> , <code>src/utls/pdf.js</code>	Mirrors backend calculations and provides PDF export helpers using <code>jspdf(-autotable)</code> .

2.3 Backend Structure (backend/src)

Path	Responsibility
<code>index.js</code>	Express entryptpoint, mounts routes, ensures tables exist before listening, logs <code>req.method/req.url</code> .
<code>config/db.js</code>	Pg Pool configured from <code>.env</code> (<code>DB_*</code> vars), logs connection lifecycle.
<code>routes/</code>	<code>authRoutes</code> , <code>testRoutes</code> , <code>statsRoutes</code> , <code>userRoutes</code> map HTTP paths to controllers; <code>/api/tests</code> uses <code>optionalAuth</code> .
<code>controllers/</code>	<code>authController</code> handles register/login/me/reset with Mailjet hooks; <code>testController</code> handles create/list/detail/delete/stats/export, using <code>vma</code> utilities and <code>TestModel</code> .
<code>middleware/</code>	<code>auth.js</code> (JWT helpers), <code>validation.js</code> (test payload guard), <code>errorHandler.js</code> .
<code>models/</code>	<code>TestModel</code> (CRUD + stats + CSV), <code>UserModel</code> (users, reset tokens, password hashing, ensures <code>user_id</code> column exists).
<code>utls/vma.js</code>	Source of truth for durations, validation, calculation, level classification.
<code>services/emailService.js</code>	Mailjet client with graceful fallback to console logging.

2.4 Database & Data Flow

- **Tables** are created through a mix of `migrations/run.js` (tests) and `UserModel.createTable()` (`users + password reset tokens + tests.user_id`).
- When guests post a test, `optionalAuth` leaves `req.user` undefined, so `testController.create` skips persistence and simply responds with computation details.
- Authenticated users receive a JWT (7-day expiration) stored in `localStorage` (`vma-token`) and attached to every request via `api.js`.
- Dashboards aggregate per-user stats through `TestModel.getStatsByUserId`, feeding Recharts visualizations.
- Exports rely on `json2csv` server side and `generateBulkPDF` client side.

3. VMA Calculation Logic

Source: `backend/src/utls/vma.js` (mirrored on the frontend).

3.1 Formulas

Let T_{total} be the nominal test duration (720 s for Cooper, 360 s for Demi-Cooper) and T_{stop} , T_{walk} represent pauses.

$$T_{\text{effective}} = T_{\text{total}} - T_{\text{stop}} - T_{\text{walk}} \quad VMA_{\text{km/h}} = \frac{\text{distance (km)}}{T_{\text{effective}}/3600}$$

Implementation excerpt:

```
// backend/src/utils/vma.js
const totalTestTime = TEST_DURATIONS[testType];
const effectiveTimeSeconds = totalTestTime - stopTimeSeconds - walkingTimeSeconds;
const effectiveTimeHours = effectiveTimeSeconds / 3600;
const distanceKm = distanceMeters / 1000;
const vma = Math.round((distanceKm / effectiveTimeHours) * 100) / 100;
```

3.2 Validation Rules

Defined in `validateInputs()` :

- `testType` must be `cooper` or `demi-cooper` .
- Distance required and > 0 .
- Stop/Walking times required, ≥ 0 , and `stop + walk < total duration` .
- Errors are enumerated and bubbled up to the API response (`400`).

3.3 Level Classification

`classifyLevel(vma)` returns `level` , `label` , `color` according to thresholds:

Level	Range (km/h)	Label
1	< 10	Beginner
2	$10 \leq vma < 12$	Intermediate
3	$12 \leq vma < 14$	Good
4	$14 \leq vma < 16$	Very Good
5	≥ 16	Excellent

3.4 Transparency

`calculateVMA` emits `steps[]` strings (step-by-step math) stored in `tests.calculation_steps` and displayed in `TestForm` + `TestDetail` + PDF exports for auditability.

4. Authentication System

4.1 Registration (`POST /api/auth/register`)

- Validates username (≥ 3 chars), email format, password complexity (≥ 6 chars, uppercase, number).
- Rejects duplicate emails/usernames via `UserModel.findBy*` .
- Hashes password with `bcryptjs` (12 salt rounds).

- Generates JWT via `generateToken(user)` (payload: `id`, `username`, `email`, expires in 7 days) and sends async welcome email.

4.2 Login (POST /api/auth/login)

- Accepts `identifier` (email or username) + password.
- Uses `UserModel.findByEmailOrUsername`, `verifyPassword`, issues JWT on success.

4.3 Session Handling

- Frontend stores token in `localStorage` as `vma-token` and keeps user info in `AuthContext`.
- Every fetch in `services/api.js` injects `Authorization: Bearer <token>`.
- Protected React routes are wrapped with `<ProtectedRoute>` which waits for `AuthContext.loading` before redirecting to `/login`.

4.4 Password Reset

- `POST /api/auth/forgot-password` always responds success to avoid enumeration, but only creates tokens for existing emails.
- `UserModel.createResetToken` invalidates previous tokens, inserts new row in `password_reset_tokens` with 1-hour expiry.
- `sendPasswordResetEmail` builds a branded HTML email with CTA + fallback plain text.
- `POST /api/auth/reset-password` verifies token (`UserModel.findValidResetToken`), enforces password rules, updates hash, and marks token as used.

4.5 Logout + Me

- Logout endpoint is stateless; frontend simply deletes the token.
- `/api/auth/me` is protected and returns sanitized user info.

5. API Endpoints

(All paths are mounted under `/api` in `backend/src/index.js`.)

Method	Path	Auth	Controller	Notes
GET	<code>/health</code>	Public	<code>inline</code>	Health probe for monitoring.
POST	<code>/auth/register</code>	Public	<code>authController.register</code>	Creates user, returns { user, token }.
POST	<code>/auth/login</code>	Public	<code>authController.login</code>	Accepts email or username.
POST	<code>/auth/logout</code>	Public	<code>authController.logout</code>	Stateless acknowledgement.
GET	<code>/auth/me</code>	Bearer required	<code>authController.me</code>	Returns current user.
POST	<code>/auth/forgot-password</code>	Public	<code>authController.forgotPassword</code>	Sends reset email if user exists.

POST	/auth/reset-password	Public	authController.resetPassword	Requires { token, password }.
POST	/tests	Optional	testController.create	Guests compute only; authenticated users persist result. Payload validated by validateTestInput.
GET	/tests	Bearer	testController.getAll	Query params: level, type, search, sortBy, sortOrder, page, limit. Returns paginated results.
GET	/tests/:id	Bearer	testController.getById	Enforces ownership.
DELETE	/tests/:id	Bearer	testController.delete	Enforces ownership.
GET	/tests/export/csv	Bearer	testController.exportCSV	Streams CSV built via json2csv, reuses filters.
GET	/stats	Bearer	testController.getStats	Aggregated stats scoped to user.
GET	/users/tests	Bearer	testController.getUserTests	Convenience alias for listing saved tests.

Common response shape:

```
{
  "success": true,
  "data": { ... },
  "pagination": { ... }
}
```

Errors follow { success: false, error | errors } per controller.

6. Email Service (Mailjet)

Implementation: backend/src/services/emailService.js .

- **Client bootstrap:** Lazily connects via node-mailjet using MJ_APIKEY_PUBLIC / MJ_APIKEY_PRIVATE . Missing credentials trigger console logging instead of failing the request.
- **From identity:** Configurable via FROM_EMAIL and FROM_NAME , defaulting to noreply@vmacalculator.com / "VMA Calculator".
- **Welcome email:** Triggered post-registration. Rich HTML template with CTA pointing to APP_URL . Example subject: "Welcome to VMA Calculator! 🧑🏻".
- **Password reset:** Triggered after creating reset token. Includes button + raw link with 1-hour expiry reminder.

- **Error handling:** Mailjet failures are logged (`console.error('Mailjet error')`) but do not reject API responses, preventing auth flow disruption.
 - **Next steps:** Mailjet account is currently blocked (401). See Troubleshooting for mitigation options.
-

7. Database Schema

7.1 Core Tables (from `backend/migrations/run.js` and `models/userModel.js`)

```
CREATE TABLE IF NOT EXISTS tests (  
  id SERIAL PRIMARY KEY,  
  first_name VARCHAR(100) NOT NULL,  
  last_name VARCHAR(100) NOT NULL,  
  age INTEGER NOT NULL CHECK (age > 0 AND age < 150),  
  gender VARCHAR(20) NOT NULL CHECK (gender IN ('male','female','other')),  
  weight DECIMAL(5,2),  
  test_type VARCHAR(20) NOT NULL CHECK (test_type IN ('cooper','demi-cooper')),  
  distance_meters DECIMAL(10,2) NOT NULL CHECK (distance_meters > 0),  
  stop_time_seconds DECIMAL(10,2) NOT NULL DEFAULT 0 CHECK (stop_time_seconds >= 0),  
  walking_time_seconds DECIMAL(10,2) NOT NULL DEFAULT 0 CHECK (walking_time_seconds  
>= 0),  
  effective_time_seconds DECIMAL(10,2) NOT NULL,  
  vma DECIMAL(6,2) NOT NULL,  
  level INTEGER NOT NULL CHECK (level BETWEEN 1 AND 5),  
  level_label VARCHAR(50) NOT NULL,  
  calculation_steps TEXT[],  
  user_id INTEGER REFERENCES users(id) ON DELETE CASCADE,  
  created_at TIMESTAMPTZ DEFAULT NOW(),  
  updated_at TIMESTAMPTZ DEFAULT NOW()  
);  
  
CREATE TABLE IF NOT EXISTS users (  
  id SERIAL PRIMARY KEY,  
  username VARCHAR(100) NOT NULL UNIQUE,  
  email VARCHAR(255) NOT NULL UNIQUE,  
  password_hash VARCHAR(255) NOT NULL,  
  created_at TIMESTAMPTZ DEFAULT NOW(),  
  updated_at TIMESTAMPTZ DEFAULT NOW()  
);  
  
CREATE TABLE IF NOT EXISTS password_reset_tokens (  
  id SERIAL PRIMARY KEY,  
  user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  token VARCHAR(255) NOT NULL UNIQUE,  
  expires_at TIMESTAMPTZ NOT NULL,  
  used BOOLEAN DEFAULT FALSE,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

7.2 Indexes

- `idx_tests_test_type` , `idx_tests_level` , `idx_tests_last_name` , `idx_tests_vma` , `idx_tests_user_id` (added via `UserModel.createTable`).
- `idx_users_email` , `idx_users_username` .
- `idx_reset_tokens_token` .

7.3 Relationships

- `users` (1) — (N) `tests` via `tests.user_id` (cascade delete ensures orphan cleanup).
- `users` (1) — (N) `password_reset_tokens` .

7.4 Seeds

`backend/seed/seed.js` populates `tests` with realistic samples (Cooper & Demi-Cooper) plus repeated athletes for progression charts.

8. Key Frontend Components

Component / Page	File	Responsibility
Header	<code>components/Header.jsx</code>	Navigation + dark mode toggle + auth dropdown. Responsive between desktop/mobile navs.
ProtectedRoute	<code>components/ProtectedRoute.jsx</code>	Blocks children until auth status resolved (shows spinner).
TestSelection	<code>pages/TestSelection.jsx</code>	Landing page with Cooper vs Demi-Cooper cards and educational copy.
TestForm	<code>pages/TestForm.jsx</code>	Complex form with validation, live preview (<code>calculateVMA</code>), guest/auth flows, toast notifications.
Results	<code>pages/Results.jsx</code>	Paginated, filterable list with CSV/PDF export, deletion actions, mobile-friendly cards.
TestDetail	<code>pages/TestDetail.jsx</code>	Detailed breakdown + PDF export per test.
Dashboard	<code>pages/Dashboard.jsx</code>	Recharts visualizations for levels, types, gender breakdown, rankings, progression lines.
Profile	<code>pages/Profile.jsx</code>	User summary, quick stats, latest tests list.
Auth Screens	<code>pages/Signup.jsx</code> , <code>Login.jsx</code> , <code>ForgotPassword.jsx</code> , <code>ResetPassword.jsx</code>	Input validation mirroring backend rules, Toast feedback, lucide icons.
API Service	<code>services/api.js</code>	Centralizes fetch calls, handles JSON/CSV responses, error normalization.

PDF utilities	utils/pdf.js	jsPDF templates for single/bulk exports with branding + tables.
State Providers	context/AuthContext.jsx, context/ThemeContext.jsx	Manage session/dark mode; consumed by header, routes, and core pages.

State management relies on React Context + hooks; data fetching is performed per page using `useEffect` + `api.*` calls with built-in loading/error/Toast states.

9. Deployment Architecture

9.1 Current Modes

- Quick Demo (Cloudflare quick tunnel):** `./scripts/start_tunnel.sh` builds the SPA, ensures backend + proxy + cloudflared are running, and exposes a temporary `https://<hash>.trycloudflare.com` URL. Ideal for rapid sharing from a local machine.
- Persistent Hosting (recommended):** Deploy the compiled frontend + backend onto a VPS, fronted by Nginx and a named Cloudflare Tunnel. PM2 keeps Node processes alive, while Cloudflare provides TLS/WAF/DDoS protection. Database stays on the same host or managed Postgres.

9.2 Infrastructure Diagram (text)

```
User → Cloudflare Tunnel (*.trycloudflare.com or vma.maryamkarim.com)
  → Nginx (Reverse Proxy, /var/www/vma-calculator)
  → PM2 (manages Node processes)
  → Node.js Backend (backend/src/index.js)
    ↘
      PostgreSQL 14 (localhost:5432)
```

9.3 Domain

- Current:** ephemeral `https://<random>.trycloudflare.com` generated per tunnel session (script reads from `cloudflared.log`).
- Target:** `vma.maryamkarim.com` mapped through Cloudflare DNS to a named tunnel using `deploy/cloudflared/config.yml`.

10. Deployment Configuration Details

10.1 Nginx

File: `deploy/nginx/vma-calculator.conf`

```
upstream vma_api {
    server 127.0.0.1:3001;
    keepalive 32;
}

server {
    listen 80;
    server_name vma.maryamkarim.com;
```

```

root /var/www/vma-calculator/current/frontend/dist;
index index.html;

add_header X-Frame-Options SAMEORIGIN always;
add_header Strict-Transport-Security "max-age=63072000; includeSubDomains;
preload" always;

location / { try_files $uri $uri/ /index.html; }
location = /healthz { proxy_pass http://vma_api/api/health; }
location /api/ {
    proxy_pass http://vma_api$request_uri;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
}

```

Deploy via `sudo ln -s` into `/etc/nginx/sites-enabled/` and reload.

10.2 Cloudflare Tunnel

File: `deploy/cloudflared/config.yml`

```

tunnel: vma-calculator
credentials-file: /home/gladius/.cloudflared/vma-calculator.json

ingress:
  - hostname: vma.maryamkarim.com
    service: http://localhost:80
  - service: http_status:404

metrics: localhost:54322

```

Run as a service: `cloudflared --config /path/config.yml --origincert ~/.cloudflared/cert.pem service install`.

10.3 PM2 Ecosystem

File: `deploy/pm2/ecosystem.config.js`

```

module.exports = {
  apps: [
    {
      name: 'vma-api',
      cwd: '/home/gladius/Codes/project/backend',
      script: 'src/index.js',
      env: { PORT: 3001, NODE_ENV: 'production', APP_URL:
'https://vma.maryamkarim.com' },
    },
    {
      name: 'vma-proxy',

```

```

    cwd: '/home/gladius/Codes/project/scripts',
    script: 'proxy.js',
    env: { PORT: 8080, API_TARGET: 'http://127.0.0.1:3001' }
  }
]
};

```

Usage: `pm2 start deploy/pm2/ecosystem.config.js && pm2 save && pm2 startup systemd` .

10.4 Environment Variables

Variable	Purpose	Where Used
DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD	PostgreSQL connection (config/db.js).	Backend.
PORT	Express listen port (default 3001).	Backend.
NODE_ENV	Toggles production error messages.	Backend + frontend build decisions.
JWT_SECRET	Signing key for jsonwebtoken.	middleware/auth.js.
MJ_APIKEY_PUBLIC, MJ_APIKEY_PRIVATE	Mailjet credentials.	services/emailService.js.
FROM_EMAIL, FROM_NAME	Email branding fallback.	emailService.
APP_URL	Used in emails + frontend links.	emailService.
API_TARGET, PORT (proxy)	Configure scripts/proxy.js.	Proxy.
CLOUDFLARED_HOME, TUNNEL_TOKEN	(optional) for Cloudflare named tunnels.	System-level.

Store secrets outside version control (`backend/.env` already gitignored).

10.5 Domain & DNS

1. Create CNAME `vma` pointing to `vma-calculator.example.com` (Cloudflare-managed). For named tunnels, Cloudflare automatically creates `CNAME <hostname> -> <tunnel-id>.cfargotunnel.com` .
2. Enable "Full (strict)" SSL mode to ensure Cloudflare trusts the origin certificate (if terminating TLS at Nginx) or rely on tunnel encryption.
3. Apply WAF rules or rate limiting as needed via Cloudflare dashboard.

10.6 Tunnel Orchestrator Script

`scripts/start_tunnel.sh` automates demo hosting:

1. Builds frontend (`npm run build`).
2. Starts backend (`node src/index.js`) if not already running; logs to `backend.log` .
3. Installs/starts Express proxy (`scripts/proxy.js` , logs `proxy.log`).

4. Ensures `cloudflared` binary exists (downloads latest `.deb` if missing).
 5. Launches quick tunnel exposing `http://localhost:8080` and polls `cloudflared.log` for the public URL.
-

11. Deployment Steps

1. **Provision server:** Ubuntu 22.04 VPS (2 vCPU / 4 GB RAM recommended), install Node.js ≥ 18 , npm, PostgreSQL 14+, Nginx, PM2, cloudflared.
 2. **Clone repo:** `git clone ... && cd project && npm install && npm run install:all`.
 3. **Configure environment:**
 - Copy `backend/.env.example` $\rightarrow$ `.env` and set DB creds, `JWT_SECRET`, Mailjet keys, `APP_URL`.
 - Create PostgreSQL database/user matching `.env` (e.g., `CREATE DATABASE vma_calculator;`).
 4. **Run migrations & seeds:**
 - `cd backend && npm run migrate` to create tables.
 - Optionally `npm run seed` for demo data.
 5. **Build frontend:** `cd frontend && npm run build` (outputs to `frontend/dist`). Sync `dist/` to `/var/www/vma-calculator/current/frontend/dist` if deploying outside repo.
 6. **Install services:**
 - Copy `deploy/nginx/vma-calculator.conf` into `/etc/nginx/sites-available/`, enable, and reload Nginx.
 - Start PM2 with `deploy/pm2/ecosystem.config.js` and enable startup.
 - Configure cloudflared using `deploy/cloudflared/config.yml` (named tunnel) or run quick tunnel script for temporary demos.
 7. **Manage database migrations going forward:** rerun `npm run migrate` after schema changes; use `psql` for verifying indexes.
 8. **Monitoring & Logs:**
 - Backend: `tail -f /home/gladious/Codes/project/backend.log` (or `pm2 logs vma-api`).
 - Proxy: `tail -f proxy.log` (if used).
 - Cloudflared: `tail -f cloudflared.log` (quick tunnel) or `journalctl -u cloudflared` (systemd service).
 - Nginx: `sudo journalctl -u nginx -f`.
 9. **Backups:** schedule `pg_dump` for `vma_calculator` DB; rotate logs weekly.
-

12. Security Considerations

- **Network perimeter:** Cloudflare Tunnel hides the origin IP and provides WAF/DDoS filtering; ensure access tokens are revoked when unused.
- **Reverse proxy hardening:** Security headers + HSTS already defined in Nginx config; consider CSP when finalizing hosting.
- **Database:** Column-level constraints and CHECK clauses enforce bounds; restrict PostgreSQL to localhost or a private subnet. Use strong passwords and optionally enable SSL.
- **Secrets:** `JWT_SECRET`, DB creds, and Mailjet keys must stay outside version control. Leverage environment management (e.g., `pm2 set env VAR=value`) or `.env` with proper permissions.
- **Authentication:** JWT expires in 7 days; consider refresh tokens for longer-lived sessions if needed. Enforce HTTPS-only cookies on any future cookie storage.

- **Input validation:** Both backend (`validateInputs` , `validateTestInput`) and frontend enforce correct data ranges, minimizing invalid DB writes.
- **Email abuse:** Forgot-password endpoint masks user existence. Rate limiting (e.g., via Cloudflare Rules) is recommended to prevent brute force.
- **Logging:** Request logger prints method + path; avoid logging sensitive payloads. Ensure logs rotate to prevent disk exhaustion.

13. Troubleshooting

Symptom	Log / Signal	Root Cause	Resolution
Mailjet responses 401 Unauthorized.	<code>backend.log: Mailjet error: Unauthorized.</code>	Account currently blocked (per prior support ticket).	Contact Mailjet support or switch provider (SendGrid/Postmark). Meanwhile, emails fall back to console logging.
Cloudflare tunnel prints URL but requests fail with 502.	<code>cloudflared.log + HTTP 502.</code>	Proxy or backend not running / port mismatch.	Check <code>proxy.log</code> and <code>backend.log</code> , ensure <code>API_TARGET</code> matches backend port, restart via <code>scripts/start_tunnel.sh</code> .
<code>/api/tests</code> returns 401 even when logged in.	API response Authentication required.	Missing/expired JWT in <code>localStorage</code> .	Re-login; ensure browser accepts <code>localStorage</code> ; clear token via dev tools if corrupted.
Nginx serves 404 for SPA routes.	<code>/var/log/nginx/vma-calculator.error.log.</code>	<code>try_files</code> misconfigured or frontend/dist missing.	Rebuild frontend, verify root path, reload Nginx.
<code>pm2 resurrect</code> fails after reboot.	<code>pm2 ls</code> empty.	Startup script not registered.	Run <code>pm2 startup</code> (as root), <code>pm2 save</code> , and confirm systemd unit exists.

14. Developer Credits

- **Developed by:** Maryam Karim
- **Contact:** maryamkarimbac@gmail.com
- **Phone:** +224 70 24 91 230
- **Repository:** `/home/gladius/Codes/project`

This document is generated directly from the source tree to ensure every section references real code and configuration files. Convert to PDF via `npx md-to-pdf docs/technical-documentation.md` for sharing.